# Instituto Federal de Educação, Ciência e

**Tecnologia de São Paulo - IFSP**
**Campus Jacareí**
**Abril de 2026**
**Qualidade e Teste de Software- JCRTQSW**

**Lista de Exercícios 08**

**Fabio Vivarelli**
**João Pedro Veríssimo**
**João Vitor Gimenes**
**Juan Santos Trigo Nasser**
**Nathan Henrique**

**Sumário**

**Instituto Federal de Educação, Ciência e.....1**
**Exercício A.....3**
**Exercício B.....5**
**Exercício C.....8**
**Exercício D.....11**
**Exercício E.....13**

# Exercício A

A classe ExA apresentou 90% de cobertura de instruções e 100% de cobertura de ramos, demonstrando que todos os caminhos condicionais foram devidamente explorados pelos testes. Isso indica que os cenários envolvendo valores de x e y, incluindo casos onde y == 0, x == 0 e o fluxo normal do laço, foram corretamente contemplados. A pequena quantidade de instruções não cobertas pode estar associada a detalhes específicos de execução, mas não compromete a validação da lógica principal. De modo geral, os testes para esta classe são consistentes e atingem plenamente os objetivos de cobertura de decisão.

ExA.java:

```java
package com.exercicios;

public class ExA {

    public static void compute(int x, int y) {
        if (y == 0)
            System.out.println("y is zero");
        else if (x == 0)
            System.out.println("x is zero");
        else {
            for (int i = 1; i <= x; i++) {
                if (i % y == 0)
                    System.out.println(i);
            }
        }
    }

}
```

TestAll.java

```java
package com.exercicios;

import org.junit.Test;
import static org.junit.Assert.*;

public class TestAll {

    // ======================
    // ExA
    // ======================
    @Test
```

```java
    public void testA_yZero() {
        ExA.compute(5, 0);
    }

    @Test
    public void testA_xZero() {
        ExA.compute(0, 2);
    }

    @Test
    public void testA_normal() {
        ExA.compute(5, 2);
    }
```

ExA.java.html:

lista10 > com.exercicios > ExA

## ExA

| Element | Missed Instructions | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods |
|---|---|---|---|---|---|---|---|---|---|---|
| ExA() | | 0% | | n/a | 1 | 1 | 1 | 1 | 1 | 1 |
| compute(int, int) | | 100% | | 100% | 0 | 5 | 0 | 8 | 0 | 1 |
| Total | 3 of 30 | 90% | 0 of 8 | 100% | 1 | 6 | 1 | 9 | 1 | 2 |

# Exercício B

A classe ExB obteve 92% de cobertura de instruções e 100% de cobertura de ramos, o que demonstra que os testes foram eficazes na validação do algoritmo de busca binária. Todos os caminhos possíveis do fluxo de decisão incluindo busca bem-sucedida, valores menores e maiores foram exercitados. A leve perda na cobertura de instruções pode indicar a ausência de cenários mais específicos, como variações extremas do vetor, mas não impacta significativamente a qualidade dos testes. O resultado evidencia uma boa compreensão dos diferentes comportamentos do algoritmo.

```java
package com.exercicios;

public class ExB {
    public static int lookup(int vector[], int key, int n) {
        int low = 0;
        int high = n - 1;

        while (low <= high) {
            int medium = (low + high) / 2;

            if (vector[medium] == key)
                return medium;
            else if (vector[medium] < key)
                low = medium + 1;
            else
                high = medium - 1;
        }
        return -1;
    }
}
```

```java
    // ======================
    // ExB
    // ======================
    @Test
    public void testB_found_middle() {
        int v[] = {1,2,3,4,5};
        assertEquals(2, ExB.lookup(v, 3, v.length));
    }
```

```java
@Test
public void testB_notFound() {
    int v[] = {1,2,3,4,5};
    assertEquals(-1, ExB.lookup(v, 10, v.length));
}

@Test
public void testB_first() {
    int v[] = {1,2,3,4,5};
    assertEquals(0, ExB.lookup(v, 1, v.length));
}

@Test
public void testB_last() {
    int v[] = {1,2,3,4,5};
    assertEquals(4, ExB.lookup(v, 5, v.length));
}

@Test
public void testB_singleElement() {
    int v[] = {7};
    assertEquals(0, ExB.lookup(v, 7, v.length));
    assertEquals(-1, ExB.lookup(v, 1, v.length));
}
```

lista10 > com.exercicios > ExB

## ExB

| Element | Missed Instructions | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods |
|---|---|---|---|---|---|---|---|---|---|---|
| ExB() | | 0% | | n/a | 1 | 1 | 1 | 1 | 1 | 1 |
| lookup(int[], int, int) | | 100% | | 100% | 0 | 4 | 0 | 11 | 0 | 1 |
| Total | 3 of 42 | 92% | 0 of 6 | 100% | 1 | 5 | 1 | 12 | 1 | 2 |

# Exercício C

A classe ExC apresentou 93% de cobertura de instruções e 65% de cobertura de ramos, evidenciando uma discrepância entre execução do código e exploração completa das decisões. Embora quase todas as instruções tenham sido executadas, parte significativa dos ramos condicionais não foi coberta. Isso se deve à complexidade do método, que envolve múltiplas condições compostas, estruturas de repetição e decisões encadeadas. Além disso, o uso de expressões booleanas combinadas (como condition || ...) exige testes específicos para cada combinação de valores, o que nem sempre foi contemplado. Esse resultado indica que, apesar da boa execução geral, ainda há necessidade de ampliar os testes para cobrir todas as variações possíveis das condições.

```
public class ExC {

    public void example1(boolean condition, String string1, ErrorCodes errno, char
input[]) {
        int p[] = null;
        char string2[] = null;
        String message[] = new String[3];

        if (condition) {
            p = new int[2];
            p[0] = 1;
        }

        if (p != null)
            p[1] = 0;

        if (condition)
            System.out.println("They are equal");

        message[0] = "Permission denied";
        message[1] = "No such device";
        message[2] = "No such file or directory";

        System.out.println(message[errno.getErrorNum()]);

        char output[] = new char[100];

        for (int i = 0; i < input.length && i < output.length; i++) {
            output[i] = input[i];
            if (input[i] == '\0')
                break;
        }
    }
}
```

```java
// ====================
// ExC
// ====================

@Test
public void testC_conditionTrue_printsEqual() {
    ExC c = new ExC();

    String output = captureOutput(() ->
        c.example1(true, "abc", ErrorCodes.EACCES, new char[]{'a','\0'})
    );

    assertTrue(output.contains("They are equal"));
}

@Test
public void testC_conditionFalse_noEqualPrint() {
    ExC c = new ExC();

    String output = captureOutput(() ->
        c.example1(false, "abc", ErrorCodes.EACCES, new char[]{'a','\0'})
    );

    assertFalse(output.contains("They are equal"));
}

@Test
public void testC_allErrorMessages() {
    ExC c = new ExC();

    String out1 = captureOutput(() ->
        c.example1(true, "a", ErrorCodes.EACCES, new char[]{'x','\0'})
    );
    assertTrue(out1.contains("Permission denied"));

    String out2 = captureOutput(() ->
        c.example1(true, "a", ErrorCodes.ENODEV, new char[]{'x','\0'})
    );
    assertTrue(out2.contains("No such device"));

    String out3 = captureOutput(() ->
        c.example1(true, "a", ErrorCodes.ENOENT, new char[]{'x','\0'})
    );
    assertTrue(out3.contains("No such file or directory"));
}
```

lista10 > com.exercicios > ExC

# ExC

| Element | Missed Instructions | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods |
|---|---|---|---|---|---|---|---|---|---|---|
| example1(boolean, String, ErrorCodes, char[]) | | 92% | | 65% | 6 | 11 | 0 | 24 | 0 | 1 |
| ExC() | | 100% | | n/a | 0 | 1 | 0 | 1 | 0 | 1 |
| Total | 7 of 101 | 93% | 7 of 20 | 65% | 6 | 12 | 0 | 25 | 0 | 2 |

# Exercício D

A classe ExD apresentou 88% de cobertura de instruções e 100% de cobertura de ramos, demonstrando que todas as decisões foram devidamente testadas. Os testes contemplaram cenários relevantes, como valores pares, ímpares e nulos, garantindo a validação completa da lógica condicional do método. A pequena parcela de instruções não cobertas pode estar associada a detalhes do fluxo iterativo, mas não compromete a cobertura de decisões. Trata-se de um exemplo de código bem testado e com boa estrutura para validação.

```java
package com.exercicios;

public class ExD {
    public static Integer m(int i, Integer obj) {
        int s = 0;

        if (obj == null)
            return null;

        if (i % 2 == 1)
            i++;

        for (; i > 0; i--)
            s += obj;

        return s;
    }
}
```

```java
    @Test
    public void testD_oddBecomesEven() {
        assertEquals(Integer.valueOf(8), ExD.m(3, 2));
    }

    @Test
    public void testD_even() {
        assertEquals(Integer.valueOf(4), ExD.m(2, 2));
    }

    @Test
    public void testD_zero() {
        assertEquals(Integer.valueOf(0), ExD.m(0, 2));
    }

    @Test
    public void testD_null() {
        assertNull(ExD.m(5, null));
    }
```

lista10 > com.exercicios > ExD

# ExD

| Element | Missed Instructions | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods |
|---|---|---|---|---|---|---|---|---|---|---|
| ExD() | | 0% | | n/a | 1 | 1 | 1 | 1 | 1 | 1 |
| m(int, Integer) | | 100% | | 100% | 0 | 4 | 0 | 8 | 0 | 1 |
| Total | 3 of 27 | 88% | 0 of 6 | 100% | 1 | 5 | 1 | 9 | 1 | 2 |

# Exercício E

A classe ExE apresentou os resultados mais baixos, com 69% de cobertura de instruções e 60% de cobertura de ramos, indicando que diversos caminhos do código não foram explorados. Isso sugere que os testes não contemplaram todas as combinações possíveis das condições envolvendo os limites do intervalo (a, b e x). Em particular, casos de fronteira — como x == a ou x == b — e combinações que ativam múltiplas condições simultaneamente podem não ter sido testados. A estrutura condicional aninhada exige uma maior variedade de cenários para garantir cobertura completa, o que ainda não foi atingido.

```java
package com.exercicios;

public class ExE {
    public static String contains(int x, int a, int b) {
        if (a <= b) {
            if (a <= x && x <= b)
                return "inside";

            if (x <= a || x >= b)
                return "outside";
        } else {
            return "empty";
        }
        return "none";
    }
}
```

```java
@Test
    public void testE_inside() {
        assertEquals("inside", ExE.contains(5,1,10));
    }

    @Test
    public void testE_outside_low() {
        assertEquals("outside", ExE.contains(0,1,10));
    }

    @Test
    public void testE_outside_high() {
        assertEquals("outside", ExE.contains(11,1,10));
    }

    @Test
    public void testE_empty() {
        assertEquals("empty", ExE.contains(5,10,1));
    }

    @Test
    public void testE_boundaryA() {
        assertEquals("inside", ExE.contains(1,1,10));
    }

    @Test
    public void testE_boundaryB() {
        assertEquals("inside", ExE.contains(10,1,10));
    }
}
```

lista10 > com.exercicios > ExE

# ExE

| Element | Missed Instructions | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods |
|---|---|---|---|---|---|---|---|---|---|---|
| contains(int, int, int) | | 78% | | 60% | 3 | 6 | 1 | 7 | 0 | 1 |
| ExE() | | 0% | | n/a | 1 | 1 | 1 | 1 | 1 | 1 |
| Total | 8 of 26 | 69% | 4 of 10 | 60% | 4 | 7 | 2 | 8 | 1 | 2 |